

## 1 Problema leírása

Adott egy  $N$  hosszúságú  $a[0..N-1]$  magasságsorozat, amely egy 1D domborzatot ír le. Egy pozíció  $i$  **csúcs**, ha  $a[i-1] < a[i] > a[i+1]$ . Minden csúcsra ki kell számítani a **prominenciáját** (kiemelkedőségét): a csúcs magassága és a tőle egy magasabb csúcs felé eső legmélyebb völgy magasságának különbsége. Ha nincs magasabb csúcs az adott irányban, a prominencia a csúcs magassága (a tengerszinttől a csúcsig).

A feladat két részfeladatot tartalmaz:

- $C = 1$ : csak a csúcsok számát kell kiírni.
- Általános  $C \neq 1$ : minden csúcs prominenciáját kell meghatározni és növekvő index sorrendben kiírni.

## 2 Alapvető megfigyelések

- Csak szigorú emelkedő-csökkenő pontok minősülnek csúcsnak. Az egymás után azonos magasságú szakaszok lényegtelenek – a kettőzött értékeket törölhetjük.
- A prominencia mindkét irányban függetlenül számítható:
  - *jobb oldali prominencia*: az első nála magasabb csúcs felé eső legalacsonyabb pontig mért különbség;
  - *bal oldali prominencia*: ugyanígy a balra eső magasabb csúcs felé.
- A végső prominencia a két érték közül a **kisebbik**, mert a legmélyebb völgy mindkét oldalról dönt.
- A feladat lényegileg egy „vízfeltöltés” probléma: ha vizet engedünk a hegységbe, minden csúcs körül a vízszint a hozzá tartozó legalacsonyabb nyeregpontra emelkedik.

## 3 A megoldás alapötlete – monoton csúcs-verem

Az 1. passzolás balról jobbra haladunk, miközben egy **monoton csökkenő veremben** tároljuk a még „élő” csúcsokat (azokat, amelyeknek jobbra még nem volt magasabb szomszédjuk).

Minden pozíció  $i$ :

- Ha  $a[i]$  csúcs, a veremből sorban kivesszük (pop) az összes nála kisebb vagy egyenlő magasságú csúcsot – ezek véglegesítődnek, mert tőlük jobbra egy magasabb akadály van.
- Ha a verem nem üres, a verem tetején lévő csúcs alatt található legmélyebb völgyet (`valley[top]`) használjuk a jobb oldali prominencia kiszámításához:

$$\text{prom}[0][i] = a[i] - \text{valley}[\text{top}].$$

- Az új csúcsot betesszük a verembe és kezdeti értéként beállítjuk a hozzá tartozó völgyet (`valley[top] = a[i]`).
- Ha  $a[i]$  nem csúcs, de a verem nem üres, frissítjük a legmélyebb völgyet: ha  $a[i] < \text{valley}[top]$ , akkor  $\text{valley}[top] = a[i]$ .

A passzus végén minden csúcs **jobb oldali prominenciája** megvan.

A **bal oldali prominencia** ugyanezzel a módszerrel számolható, ha a tömböt tükrözzük (megfordítjuk), majd újra végrehajtjuk a passzust.

Végül minden csúcs végső prominenciája a két irányú érték minimuma.

## 4 Algoritmus részletezése

Az alábbiakban a megoldás fő lépéseit mutatjuk be a mellékelt C++ kód alapján.

### 4.1 A duplikált elemek eltávolítása

Beolvasás után egy ciklus törli az egymás után ismétlődő azonos magasságú elemeket, így a törusz csak szigorúan monoton szakaszokkal dolgozik.

### 4.2 Speciális eset $C = 1$

Ha a feladat csak a csúcsok számát kéri, egyszerűen megszámloljuk az  $i$  ( $1 \leq i \leq N - 2$ ) indexeket, ahol  $a[i - 1] < a[i] > a[i + 1]$ , és kiírjuk az eredményt.

### 4.3 Bal→Jobb passzus (első verem)

```
for (int d = 0; d <= 1; ++d) {
    top = -1; // ures verem
    for (int i = 1; i < N-1; ++i) {
        if (a[i-1] < a[i] && a[i] > a[i+1]) { // csucs
            prom[d][i] = a[i];
            while (top >= 0 && a[peak[top]] <= a[i]) popTop();
            if (top >= 0 && a[peak[top]] > a[i])
                prom[d][i] = a[i] - valley[top];
            peak[++top] = i;
            valley[top] = a[i];
        }
        if (top >= 0 && valley[top] > a[i])
            valley[top] = a[i];
    }
    // ... tukroz es megforditas ...
}
```

A `prom[0][i]` a jobb oldali prominencia, `prom[1][i]` a bal oldali (a tükrökép miatt) értéket tárolja.

## 4.4 Tükrözés

Az első passzus után a magasság tömböt megfordítjuk, hogy ugyanazt a kódot használhassuk a bal oldali prominencia kiszámítására:

```
for (int i = 0; i < N / 2; ++i) {
    long aux = a[i];
    a[i] = a[N-1-i];
    a[N-1-i] = aux;
}
```

## 4.5 Végleges érték

Minden csúcsra a végső prominencia a két irányú érték közül a kisebb:

```
for (int i = 1; i < N-1; ++i)
    if (a[i-1] < a[i] && a[i] > a[i+1]) {
        if (prom[0][i] > prom[1][N-1-i])
            prom[0][i] = prom[1][N-1-i];
        printf("%ld ", prom[0][i]);
    }
printf("\n");
```

## 5 Idő- és memóriakomplexitás

- Minden elemet legfeljebb egyszer teszünk a verembe és egyszer távolítjuk el belőle  $\rightarrow O(N)$  művelet.
- Két passzus és egy tükrözés  $\rightarrow O(N)$  idő.
- Öt tömb hossza  $N$  (magasságok, csúcsindexek, völgymélységek, két prominencia tömb)  $\rightarrow O(N)$  memória.

## 6 Implementációs megjegyzések

- A `long` típus elegendő mind a bemenet méretéhez ( $N \leq 10^6$ ), mind a prominencia értékének tárolásához.
- A duplikált magasságok törlése egyszerűen csökkenti a tömb méretét, de ügyelni kell a ciklus indexének frissítésére (`i-`; `N-`).
- A verem mindig csúcs-indexeket tartalmaz, a `valley[top]` a hozzá tartozó legmélyebb pont pillanatnyi értéke.
- Ha egy csúcsnak nincs nála magasabb szomszédja egyik oldalon sem, a prominenciaértéke a csúcs magassága marad – az algoritmus ezt automatikusan kezeli, mert a verem üres marad.

- A `popTop` eljárás a verem csúcsának törlése mellett frissíti a mélyebb völgyet is: ha a törlendő csúcs alatt mélyebb völgy van, az a szülő csúcs alá kerül.

## 7 Tesztelési gondolatok

- Triviális teszt: egyetlen csúcs a sorozatban – a program a teljes magasságot adja vissza.
- Több azonos magasságú szomszédos pont: a duplikált elemek eltávolítása nélkül a csúcs definíciója nem teljesülne.
- Hegymenetű és völgymentű szakaszok váltakozása: ellenőrizni kell, hogy a verem helyesen kezeli a csúcsok sorozatát.
- Nagyon nagy  $N$  (pl.  $10^6$ ) és véletlenszerű magasságok: a program  $O(N)$  futásideje garantált.

## 8 Összefoglalás

A prominencia kiszámítása 1D domborzaton hatékonyan megoldható egy **monoton csúcs-verem** segítségével. A `bal→jobb` és `jobb→bal` passzusokkal a legmélyebb völgy mindkét irányban megtalálható, és a végső érték a két érték minimuma. Az algoritmus  $O(N)$  időben és  $O(N)$  memóriában fut, ami teljesíti a feladat korlátait, és a mellékelt C++ megvalósítás ezt közvetlenül implementálja.